end of the spectrum that don't really work for us. In other words, this line limits our activity to the "sweet spot" of the sensor, making it as sensitive as we can get it to a gentle breeze. The third line takes this larger value and remaps it to a more useable range for output to our fan.

```
Serial.print("analog value = " );
Serial.println(analogValue, DEC);
```

These two lines can be used to see what values we are working with using something called the Serial Monitor, explained in more detail later in this chapter. Seeing these values is helpful in adjusting the thresholds defined at the beginning of the sketch.

```
if (analogValue > minReading && analogValue < fanStart)
analogWrite(fanPin, fanStart);
else analogWrite(fanPin, analogValue);
```

Now we reach the business end of the sketch—the part that actually does something. Our fan needs a little kick to get it going at first, so the first line checks to see if we have an analog value that we should do something with, but that might be too low for the fan to work with. In our case, we found that the value 100 is about the lowest number to guarantee that the fan would spin. So if that's true, we will send the minimum number to start the fan out to the output pin.

Otherwise, if the value is greater than or less than our `fanStart` and `minReading` values, we will send that value straight out to the output pin. If it's a small number, the fan will most likely not spin. If it's a bigger number, it will spin with greater speed up until it hits the ceiling at 255, which is full on.

```
delay(250);
```

Finally, our last line of code ends with a simple `delay()` just to slow things down some. Basically, it takes a little bit of time to get the fan up to speed, so a quarter-second pause is good for that. Now what does it all mean?

# Analog Functions

The analog functions are a little different from the digital ones. First, as we discussed earlier, they do not need to be set up using `pinMode()` beforehand. Second, we are dealing with integer numbers that range from 0 to 255 or 0 to 1023, rather than digital states like `HIGH` and `LOW`, so we need to keep that in mind while working with them. So anyway, let's proceed onward with a discussion of the analog functions.

## analogRead()

The counterpart to the `digitalRead()` function, `analogRead()` is equally as simple at first. Its syntax follows:

```
analogRead(pin)
```

When we call this function, we need to specify the analog pin that we are reading data from, in the format of A0–A5. The "A" is placed in front of the pin number to remind us that we are using one of the analog input pins, also marked on the board A0–A5. The reading returns a 10-bit value with a range of integers from 0 to1023 corresponding to the input voltage on the pin.

Also like the `digitalRead()` function, what we do with this value depends on the context in which we use the function. This includes assigning the reading to a variable to be used later in the code or to use the function in place of a variable. A common usage of this function to assign a value to a variable would appear as follows: